

MINI HIRING PIPELINE

Architecture, design decisions, search strategy, and implementation summary

Candidate: Tony Varghese

Repository: <https://github.com/tonyvargese/Hiring-Pipeline>

Assessment: Associate Gen AI Engineer take-home

Date: 29 September 2026

Design intent

A practical, testable modular monolith for one recruiter and one job. The design prioritizes correct stage transitions, append-only history, and explainable natural-language search without unnecessary infrastructure.

Submission contents

- Working React and FastAPI application with deterministic demo data.
- GitHub README with setup, decisions, assumptions, tests, and AI usage.
- AI chat log including corrections and one deliberate disagreement with AI.
- This architecture PDF with repository link and implementation trade-offs.

1. Requirements and business rules

The application manages candidates for one job through a controlled hiring pipeline. The assessment brief remains the source of truth; unspecified behavior is documented as an assumption.

Candidate management	Add candidates, group them by current stage, open details, show complete history, and calculate time in the current stage.
Progression	Advance one normal stage at a time. Skipping and backward movement are rejected.
Final outcomes	A candidate may be rejected before Hired. Hired and Rejected are terminal.
Audit trail	Every creation or transition appends an event. Existing history is never changed through the application.
Search	One search box supports typo-tolerant names, current stages, time in stage, historical movement, exclusions, combinations, ranking, and understandable errors.

Valid transition map

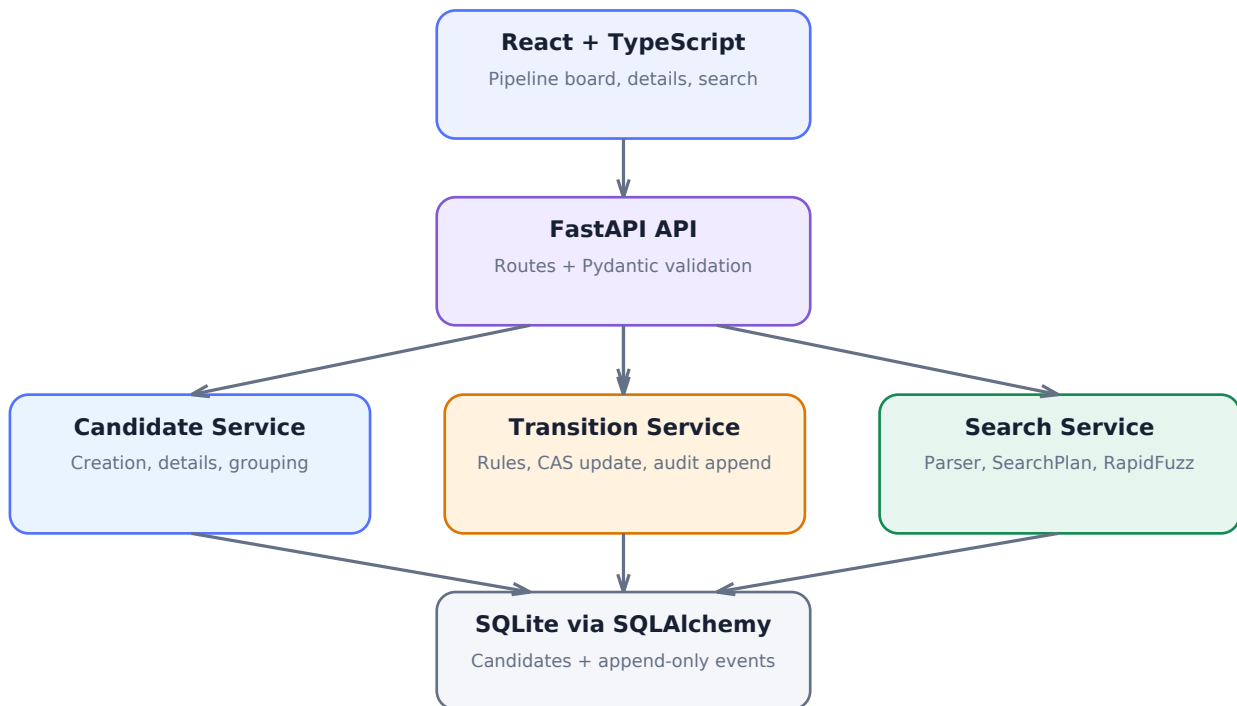
APPLIED -> SCREENING or REJECTED
SCREENING -> INTERVIEW or REJECTED
INTERVIEW -> OFFER or REJECTED
OFFER -> HIRED or REJECTED
HIRED -> no transitions
REJECTED -> no transitions

Core invariant

Candidate.current_stage equals the latest event.to_stage, and Candidate.current_stage_entered_at equals the latest event.occurred_at. Candidate updates and event insertion share one transaction.

2. Recommended system architecture

A modular monolith keeps deployment, testing, and transactions simple while preserving clear separation between HTTP handling, business rules, search interpretation, and persistence.



Responsibilities

React UI

Pipeline board, add form, candidate details, stage actions, search results, interpretation, and error states.

FastAPI routes

HTTP contracts, dependency injection, status codes, and response serialization.

Candidate service

Candidate creation, pipeline grouping, detail retrieval, duration calculation.

Transition service

Allowed-transition map, stale-stage compare-and-set, atomic update and audit append.

Search service

Deterministic parsing, date resolution, structured filters, RapidFuzz scoring, and intent-specific ranking.

SQLAlchemy + SQLite

Relational persistence with foreign keys and easy local setup; portable path to PostgreSQL.

Why not microservices?

Candidate state, audit history, and search share the same database and transaction boundaries. Separate services would add networking, deployment, and consistency complexity without improving this small assessment.

3. Data, transitions, and audit design

The model deliberately stores both the current state and every historical event. This keeps ordinary board queries simple while preserving an audit trail.

Candidate	CandidateStageEvent
id full_name normalized_name current_stage current_stage_entered_at created_at	id candidate_id from_stage (nullable) to_stage occurred_at

Transition request and compare-and-set

```
{ "from_stage": "SCREENING", "to_stage": "INTERVIEW" }
```

The backend first verifies that the stored current stage still equals from_stage. It then validates to_stage against the transition map and performs a conditional database update. A stale second request updates zero rows and returns 409 Conflict, preventing accidental double advancement.

Append-only history

- Creation writes Candidate and the initial APPLIED event together.
- Each transition uses one timestamp for current_stage_entered_at and occurred_at.
- The API exposes no update or delete operation for historical events.
- History is ordered by occurred_at and id.

Scope wording

The current implementation provides append-only history by application design. Database-level update/delete triggers can be added later for stronger enforcement.

4. Natural-language search architecture

Search is deterministic and explainable. Supported language is converted into a validated SearchPlan; raw user text never becomes SQL.

Processing pipeline

Raw query -> normalize -> recognize phrases -> build SearchPlan -> validate -> database filters -> RapidFuzz name scoring -> intent-specific ranking -> interpretation + results

Find Priya Sharam	Fuzzy candidate-name match; exact matches first, then similarity score.
Who's in Interview right now?	current_stage = INTERVIEW
Stuck in Screening for more than a week	current_stage = SCREENING and elapsed time > 7 days; longest waiting first.
Moved to Interview since Monday	An INTERVIEW event exists at or after Monday 00:00 in the application timezone.
Reached Offer but didn't get hired	An OFFER event exists and current_stage != HIRED.
Everyone except rejected candidates	current_stage != REJECTED

Example structured plan

```
{ "name": null, "current_stage": {"include": ["SCREENING"], "exclude": []}, "current_stage_age": {"operator": "GT", "seconds": 604800}, "history_predicates": [], "history_exclusions": [] }
```

Why deterministic instead of LLM-only?

The domain grammar is small. Deterministic parsing is offline, predictable, cheap, testable, and resistant to hallucinated filters. An LLM could later be an optional fallback, but only behind the same validated SearchPlan schema.

5. API contract and testing strategy

The API uses resource-oriented reads and a dedicated transition command. Structured errors distinguish missing resources, state conflicts, schema failures, and invalid search meaning.

GET /health	Application health check.
POST /api/candidates	Create candidate and initial APPLIED event.
GET /api/pipeline	Return all six stage groups, including empty groups.
GET /api/candidates/{id}	Return current state, calculated duration, allowed actions, and ordered history.
POST /api/candidates/{id}/transitions	Perform a validated, atomic stage transition.
GET /api/search?q=...	Return interpretation, SearchPlan, count, and ranked candidates.

High-value automated tests

- Candidate creation produces exactly one initial APPLIED event.
- Transition matrix covers allowed movement, skipped stages, backward movement, and terminal outcomes.
- Stale repeated transitions return 409 and append no duplicate event.
- Candidate current state remains consistent with the latest event.
- Parser corpus asserts the SearchPlan for every required query and adversarial inputs.
- Search integration tests cover typo matching, history after later advancement, elapsed-time filtering, exclusions, valid zero results, and invalid-query errors.
- Frontend lint and production build checks validate TypeScript integration.

Status code policy

404 for a missing candidate; 409 for state conflicts; 422 for request-schema or semantic search errors; 200 for a valid query with zero results.

6. Decisions, trade-offs, assumptions, and future work

The implementation favors reviewer setup, correctness, and explainability over production-scale infrastructure.

SQLite over PostgreSQL

Zero external setup and sufficient for a small local dataset. Trade-off: lower concurrency and no database-native fuzzy search.

RapidFuzz over a search engine

Excellent fit for a small candidate set. Trade-off: in-memory scoring does not scale to millions of records.

Buttons over drag and drop

Accessible and naturally restricts actions to valid transitions. Trade-off: less visual interaction.

Current state plus events

Fast stage grouping with full history. Trade-off: duplicated state requires transactional consistency and invariant tests.

No authentication

The brief describes one recruiter and does not require sign-in. Trade-off: events have no actor identity.

Explicit assumptions

- The application supports exactly one job.
- Full name is the only required candidate field; duplicate names are allowed.
- Hired and Rejected are terminal.
- Since Monday means the most recent Monday at 00:00 in Asia/Kolkata, inclusive, converted to UTC for comparison.
- More than a week means more than exactly seven elapsed days.
- Reached Offer but not Hired includes current Offer and rejected-after-Offer candidates.

With more time

- PostgreSQL, migrations, pagination, and database-native fuzzy search.
- Authentication, authorization, and actor identity on audit events.
- Database triggers for stronger append-only enforcement.
- Broader grammar and typo hints for stage keywords.
- Optional LLM fallback whose output must validate against SearchPlan.
- End-to-end browser tests, deployment, observability, and backups.

7. Submission and walkthrough plan

The walkthrough should tell one clear story rather than tour every file.

0:00-1:00	Problem, stage rules, modular-monolith overview.
1:00-3:30	Add a candidate, show Applied, advance one stage, demonstrate invalid skip.
3:30-5:00	Open details, show current-stage duration and complete history.
5:00-8:30	Run typo, current stage, stuck, since Monday, Offer-not-Hired, exclusion, and one invalid query.
8:30-10:00	Show tests and explain stale-transition protection and transaction boundary.
10:00-11:15	Explain deterministic parser versus LLM-only search; show returned SearchPlan.
11:15-12:00	Summarize trade-offs, limitations, and production next steps.

Repository

<https://github.com/tonyvargese/Hiring-Pipeline>

Run locally

Backend: `uv sync`, seed demo data, then run Uvicorn. Frontend: copy `.env.example`, `npm install`, then `npm run dev`. Full commands and prerequisites are in the repository README.

Deployment decision

Hosting is not required by the brief. A reliable local walkthrough was prioritized over a rushed deployment because a production deployment would also require persistent database hosting and production environment configuration.

End of architecture summary